

UNIDOT UNIDOT UNIDOT UNIDOT
T UNIDOT UNIDOT UNIDOT UNIDOT
OT UNIDOT UNIDOT UNIDOT UNIDOT
DOT UNIDOT UNIDOT UNIDOT UNIDOT
IDOT UNIDOT UNIDOT UNIDOT UNIDOT
NIDOT UNIDOT UNIDOT UNIDOT UNIDOT
UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
T UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
OT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
DOT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
IDOT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
NIDOT UNIDOT UNIDOT UNIDOT UNIDOT
UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
T UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
OT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
DOT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
IDOT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
NIDOT UNIDOT UNIDOT UNIDOT UNIDOT
UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
T UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
OT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
DOT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
IDOT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
NIDOT UNIDOT UNIDOT UNIDOT UNIDOT
UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
T UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
OT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
DOT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
IDOT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
NIDOT UNIDOT UNIDOT UNIDOT UNIDOT
UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
T UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
OT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
DOT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
IDOT UNIDOT UNIDOT UNIDOT UNIDOT UNIDOT
NIDOT UNIDOT UNIDOT UNIDOT UNIDOT

UNIDOT

mktable

Version 1.2
June 5, 1987

Unidot, Inc.
602 Park Point Drive
Golden, Colorado 80401
(303) 526-9263
telex: RCA 296621

UNIDOT

Copyright © 1987 by Unidot, Inc.

All rights reserved. Reproduction of this document, in whole, or in part, is strictly prohibited. For copy information, contact Unidot, Inc., 602 Park Point Dr., Golden, CO 80401.

NOTICE

The information contained in this document is subject to change without notice. Unidot Inc. makes no warranty of any kind with regard to this material, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose. Unidot Inc. assumes no responsibility for any errors that may appear in this document. Unidot Inc. makes no commitment to update nor to keep current the information contained in this document.

Introduction

I

mktable is a tool for the construction of interrelated tables, such as are used in compilers, assemblers, and other similar systems. In many ways it resembles a generalized assembler itself. In other ways it resembles a symbolic RPG. It provides for the assignment of numeric values to symbolic names (called *identifiers*) and for the general calculation of values from these and conditional testing of values.

mktable is line oriented. Although the "lines" that it can handle are quite large, this does in fact form some restrictions since most editors are not capable of conveniently handling lines that are longer than a screen width.

mktable does all of its work in one pass over the input. Normally, the first part of the input consists of defining entries (which cause internal tables to be constructed), and the latter part of the input consists of instructions on how to output this information.

Since all of the tables are constructed fully in memory the use of a single pass does not restrict use of forward references, hence cross linked tables can easily be constructed.

Input Syntax

2

Notes: Comment convention ; skips to eol Escape conventions: backslash identifier splitting: single quote

The basic style of **mktable** is free-form, line oriented text. Each line is analyzed independently, and results in either information being put into a table, being ignored, or producing output. This section describes basically that portion of the input that results in tables being constructed.

The interpretation of input is dependent on the current mode of **mktable**. The mode is set by the appearance of a special character in the first position of a line (**#**), followed by a keyword. The keyword is then optionally followed by one or more parameters. The current set of keywords recognized is the following:

```
table
output
free
dec
hex
oct
```

One other input control line is important

```
#nnn[ , nnn]
```

This is discussed in the section on table building, but in fact may appear anywhere. It does not cause a mode change.

The default mode is to ignore input. Until a suitable keyword appears, the input will merely be considered commentary, which should be used to explain the tables that follow.

Three of the above keywords, `dec`, `hex`, and `oct`, are used to change the default formats used for printing of decimal, hexadecimal, and octal constants respectively. For example, the line

```
#hex 0%xh
```

will cause hex constants to be printed with a leading 0 and a trailing `h`.

The appearance of one or more of these lines will simply cause the format to be saved, but otherwise the mode is not changed.

The appearance of the keywords, `table`, `output`, or `free`, however, does cause the mode to change.

Tables

3

A *table* may conveniently be thought of as a spreadsheet, in which there are rows and columns. Each row corresponds to exactly one line of input text. A column is terminated by one or more tabs or blanks ("white space" in UNIX jargon) or by the new line character. A comma may optionally be used between columns. The first column may optionally be a label column. If a label column is provided, **mktable** will assign numeric values to identifiers appearing in this column.

A table is started with a line of the following style:

```
#table <ucchar> <column> ...
```

where <ucchar> is one upper case alphabetic character (A-Z), and <column> is one of the following:

E	expression
S	string
C	character

An expression is made up of

decimal constants	<i>nnnnn</i>
hex constants	<i>0xhhhhh</i>
octal constants	<i>Onnnn</i>
binary constants	<i>_bbb_bbb_bbb</i>
character constants	<i>'c'</i>
symbolic names	<i><alpha><alphanumeric>...</i>

separated by operators: +, -, ×, /, %, <<, >>, !, &, ^, and ~. The set of alphanumeric characters includes the underbar (_).

Operators have no precedence, they are interpreted strictly left to right. If precedence is needed, parentheses may be used. Columns are separated by either

`<white>...<comma><white>...`

or

`<white><white>...`

An entry in a column that with the key letter S must be surrounded by double quotes (") if it contains white space or a comma. To include a double quote in such a column, the backslash may be used for an escape (\").

An entry in a column with the key letter C must be surrounded by double quotes if the character is a blank, tab, comma, or a double quote.

If the first column is an S type, it is called an *index* column or a *label* column. The symbols appearing in the first column will be assigned numeric values. The default starting value is 0. The default increment is 1. This may be changed with a line of the form

`#nnn[,nnn]`

where the first number is the next value to be assigned. The second number (if present) is the increment. If the second number is absent, the increment is 1. If the increment is negative, the value is shifted left by the increment each step. This latter feature is for the creation of bit patterns.

Symbols appearing in index columns must be unique across all tables.

Blank lines are ignored. A table continues until a keyword line appears. A new table can be started at any time.

Example: a table of registers is built by starting with the line

`#table A S S S S E`

In this example, the first S is the register name column (ie. the "name" of the register or register group), the second is the name of the register pair of

which this is a part, the third is the name of the least significant half of this register, the fourth the name of the most significant half of this register, and finally the E is a “mask” that identifier the primitive register elements of which this is composed.

The actual table would then be composed of lines like

NIL	NIL	NIL	NIL	nilreg	_0000_0000_0000_0000
R0	R01	R0R	R0L	r0	_0000_0000_0000_0001
R1	R01	R1R	R1L	r1	_0000_0000_0000_0010

How this table may be used will be described in the output section.

Output

4

After all tables are read, we are ready for output. There are several forms, generally started with something of the form:

#output [*filename*]

If the filename is missing, output continues to the file currently open as **stdout**. If filename is present, **stdout** is closed and reopened (truncated) with the filename specified.

Text following the **#output** line is copied line for line to the output with certain substitutions made:

%x (<i><expr></i>)	output the value of an expression in hex format (0xnnnn)
%s (<i><string desig></i>)	output in string format: ". . . "
%c (<i><expr></i>)	output in char format: '.'
%d (<i><expr></i>)	decimal
%o (<i><expr></i>)	octal

where the expression is composed of constants and or references of the following form:

identifier.nn

which indicates to find the value in column *nn* of the line with index *identifier*, or of the form

identifier.nn.mm

in which case the expression is repeated for each of the columns, separated by the token immediately following the terminating ")". In repeated columns,

the designator "\$" stands for the current line. As an abbreviation, *identifier* is equivalent to *identifier*. 0. Generally, the simple appearance of an *identifier* that is an index is equivalent to %d (*identifier*).

A string column when output as a string will have an ordinal assigned if the %u form is used. This is for the purpose of assigning a unique number to all identical strings. A string column when output as a value will have that number used. If no ordinal has been assigned, an error message will result. This facility allows for the construction of tables of strings in which each string will only appear one time.

If a line in an output section begins with "*" followed by a single upper case character, that line will be repeated for each source line in the referenced table.

In the output section a line beginning with "A" calls for iterating over this table.

Iteration

When iterating over a table, if the "*<char>" is followed by a <digit>, then we select only the first instance of the designated column and assign that column a value as we do so. This permits the assignment of numbers to symbols that do not appear in an index column.

Register Names

For the above mentioned register table, we output the register names in the following way:

```
char rgnames[] = {
    *A4    %s($.4),
    0 };;
```

then we output the table itself with:

```
struct rgt rgtab[] = {
    *A    %d($.1), %d($.2), %d($.3), %x($.5), %d($.4),
    0,0,0,0,0 };;
```

Free form output

A “free form” section is started with a line of the form

```
#free [filename]
```

In this section, we break apart all lines and any string that matches one of the table indexes will be replaced with its table number. To suppress the unwanted translation of a string that just happens to match a line label, precede the name with a backslash. Alternatively the ``` character (ASCII 140₈) may be used to split an item, since it is otherwise ignored (unless preceded by a backslash).